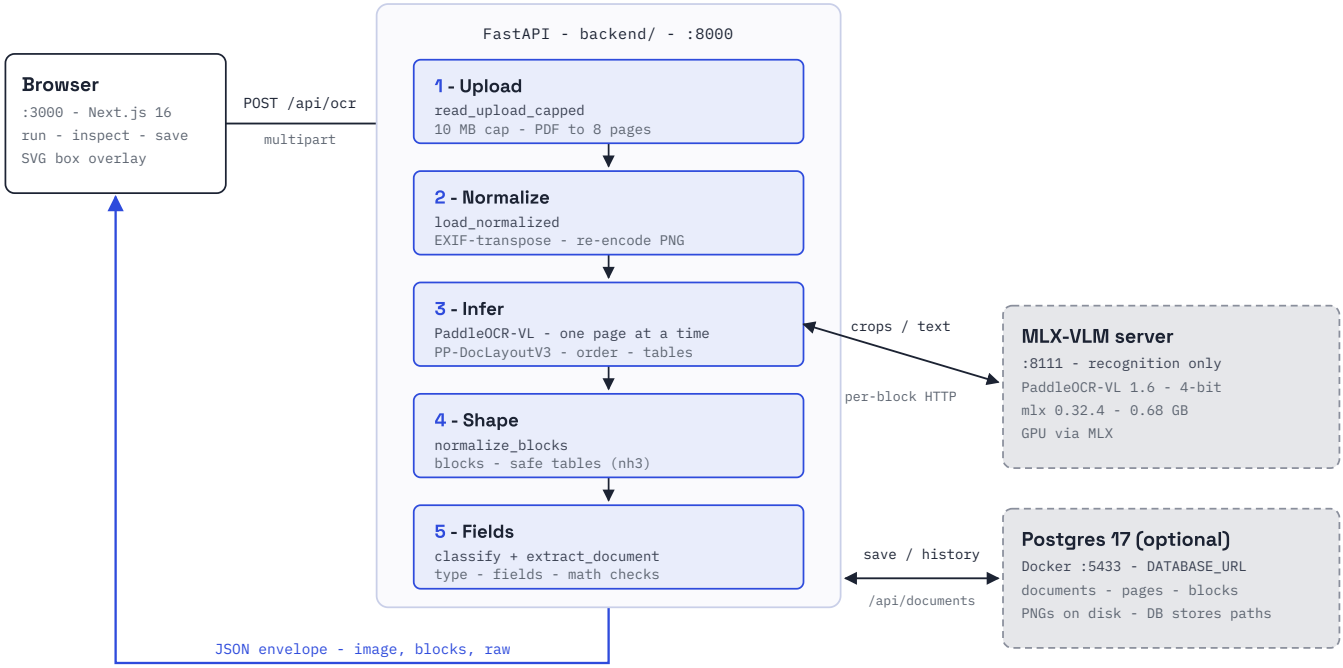
10 Invoice OCR Viewer

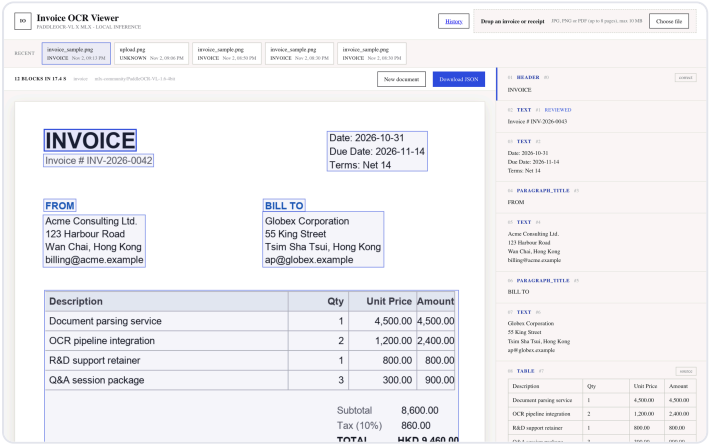
how it works - recognition on the GPU, everything on this Mac · github.com/Ethanlam123/invoice_ocr

---

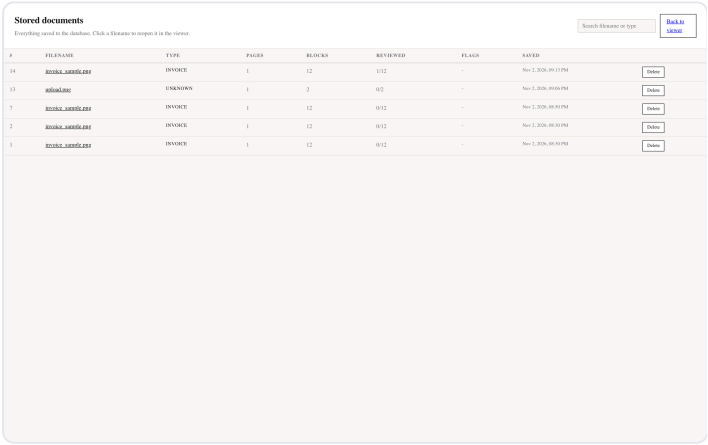


One request, end to end - the backend does everything except recognition, which it hands to the MLX server. Blue = backend stages, blue line = the response, dashed = its own process (Postgres optional). Nothing is stored unless you press Save.

01 What it looks like



The viewer: click a block on either side and both sides highlight it; REVIEWED marks a corrected block.



The history page: every saved document, searchable; click a filename to reopen it.

02 How a request flows

The backend handles everything between upload and JSON. Inference runs one page at a time behind a lock - no queue service.

1. **Upload** - `read_upload_capped` - streamed in 1 MB chunks, cut off at 10 MB; a too-big file fails before it is held in memory. PDFs split into pages (up to 8); every page runs separately.
2. **Normalize** - `load_normalized` - read by content, not by extension. EXIF-rotated, re-encoded as PNG; those exact bytes are what inference sees.
3. **Infer** - `RealPipeline` - check the MLX server is up, then run the pipeline: layout, then recognition block by block on the GPU. The model loads once; temp files are always cleaned up.
4. **Shape** - `normalize_blocks` - raw output becomes plain JSON: blocks in reading order, table HTML sanitized against an allowlist, temp-file paths stripped.
5. **Fields** - `classify + extract_document` - keyword rules name the type (invoice, receipt, PO); regex pulls number, dates and money fields; arithmetic checks line items and the total. A failed check flags `needs_review` - amounts are never auto-corrected.

03 Why the boxes line up

- The backend normalizes first; those exact PNG bytes go to the pipeline and come back as the displayed image - so every `block_bbox` is already in image pixels.
- One SVG with `viewBox="0 0 width height"` sits on top of the `img` and scales with it - boxes stay put at any size.
- Rotation and unwarping are off - nothing invisible sits between a box and its pixels.

04 Trust boundaries

- Uploads are validated by decoding and capped while streaming; CORS allows only localhost.
- Block text renders as text, never HTML; table HTML passes an nh3 allowlist server-side.
- Selection is keyed on stable `block_id`s, never on text - documents repeat text.
- A failed run clears the previous result - you never see stale output.

05 Storage and review - optional

- Nothing is stored until you press **Save to history**: one transaction writes document, pages, blocks and extracted fields; PNGs in `data/documents/<id>/`, the DB keeps paths.
- **History** (`/history`) - everything saved, searchable, with review counts and delete; `/?doc=<id>` reopens a document.
- **Corrections** - a fix is stored beside the original OCR text, never over it; reads show the fix, the original stays.

06 Stack and measured numbers

Measured on the demo machine (Apple M2, 16 GB) - no claims beyond them.

LAYER	TECHNOLOGY	ROLE
Viewer	`Next.js 16.4.3 - React 19.3`	upload - run - inspect - correct - save, one screen
API	`FastAPI 0.162 - Python 3.11 (uv)`	validates, orchestrates; one envelope per response
Layout	`paddleocr 3.7.0 - paddlepaddle 3.2.1 (CPU)`	PP-DocLayoutV3: regions, order, tables
Recognition	`mlx-vlm 0.7.11 - mlx 0.32.4`	`PaddleOCR-VL-1.6-4bit` (0.68 GB), recognition only
Storage	`Postgres 17 (Docker) - psycopg pool`	saved runs + corrections; PNGs on disk, DB keeps paths
Latency	**12.6 - 21.4 s** per page	12 - 13 blocks per invoice; ~20 s one-time init
Tests	**87 + 8 + 15**	pytest (17 opt-in SQL), vitest, Playwright mocked

Everything runs locally - no cloud APIs, no accounts; Postgres is optional, for saved history. Generated 2026-11-03.